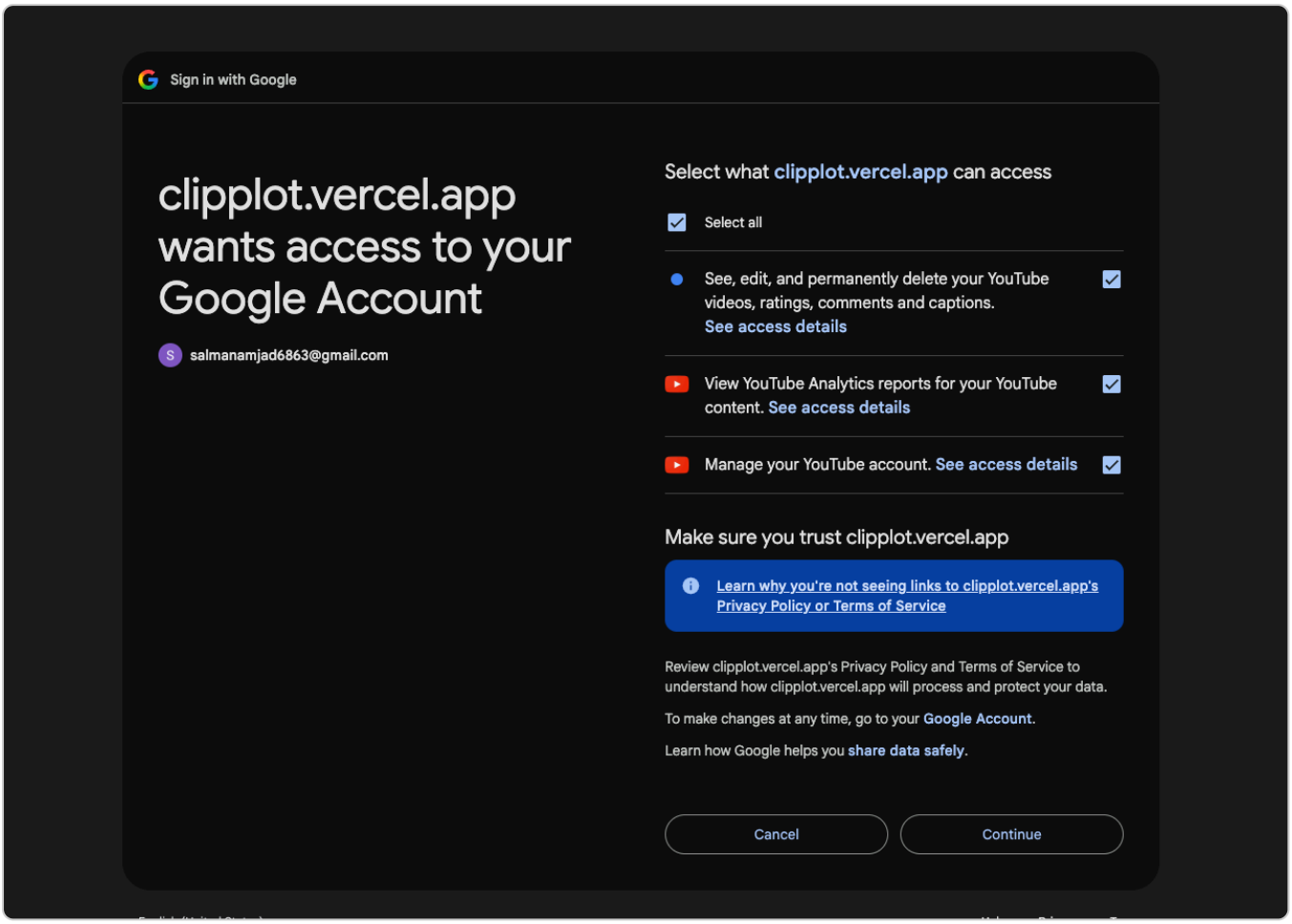
ClipPilot — Conditional Evidence (Project #1)

https://clipplot.vercel.app · Google Cloud project 878155465105 · Use case: Video Uploading & Account Management (internal tool for our own channels)

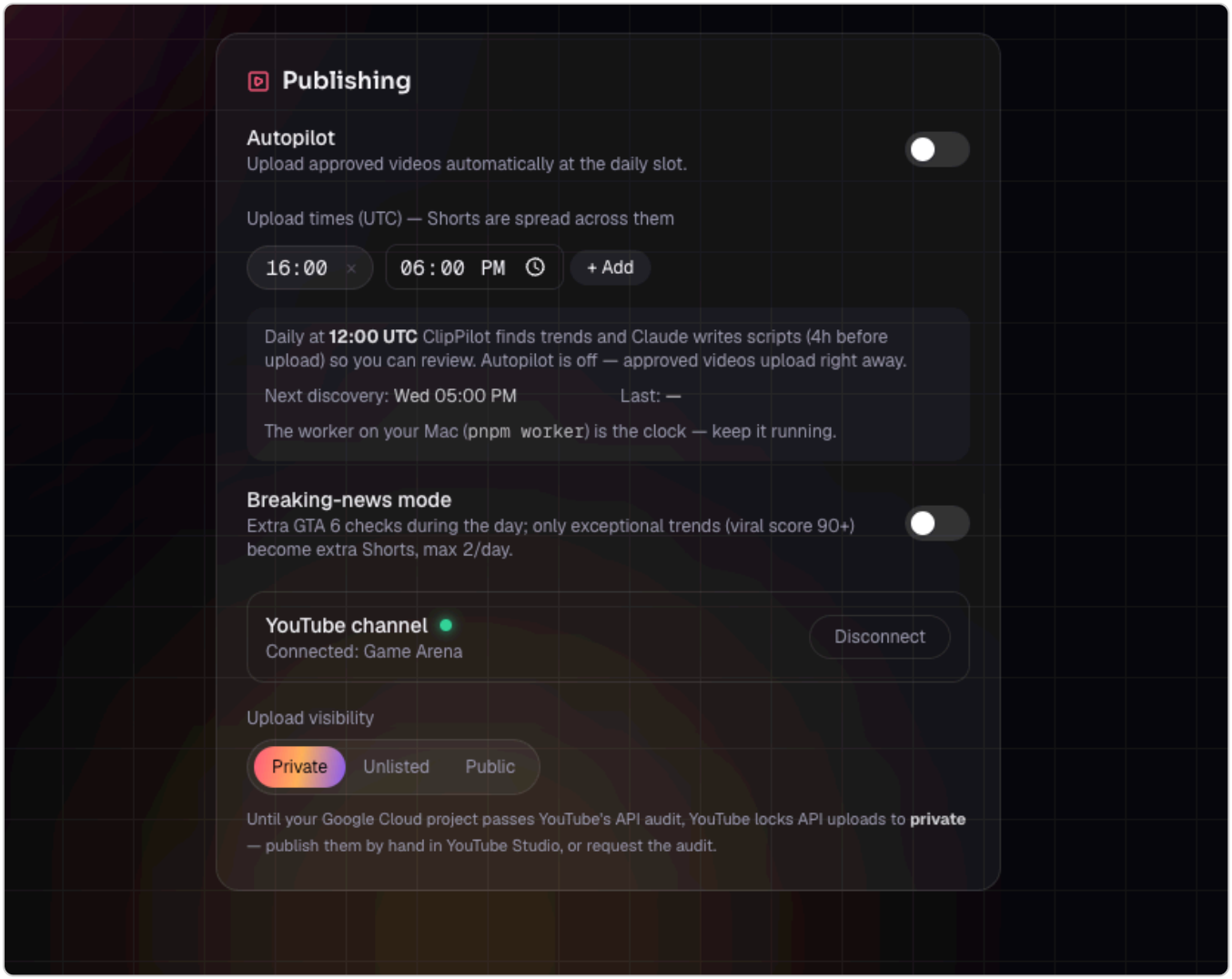
a)	OAuth flow: consent screen + scopes, connected state, revocation in-app and in Google Account
b)	Upload interface: review/edit screen and library of uploaded videos
c)	Player / Embed: **not applicable** — ClipPilot has no player or embed; it only uploads to our own channels.
d)	Dashboard / features: dashboard, trend research, settings
Upload script	Complete working upload script (videos.insert, resumable) — final pages. It uploaded test video **youtu.be/ogOBVsF5twA** (private) to our channel “Game Arena” and added it to the “GTA 6 Shorts” playlist.

Upload flow: a person reviews each draft and clicks Approve (choosing Private / Unlisted / Public) → the render worker calls `uploadToYouTube()` below with the stored OAuth refresh token → thumbnails.set and playlistItems.insert → the YouTube link appears in the Library. Nothing is uploaded without human approval.

a) OAuth flow — consent screen and scopes

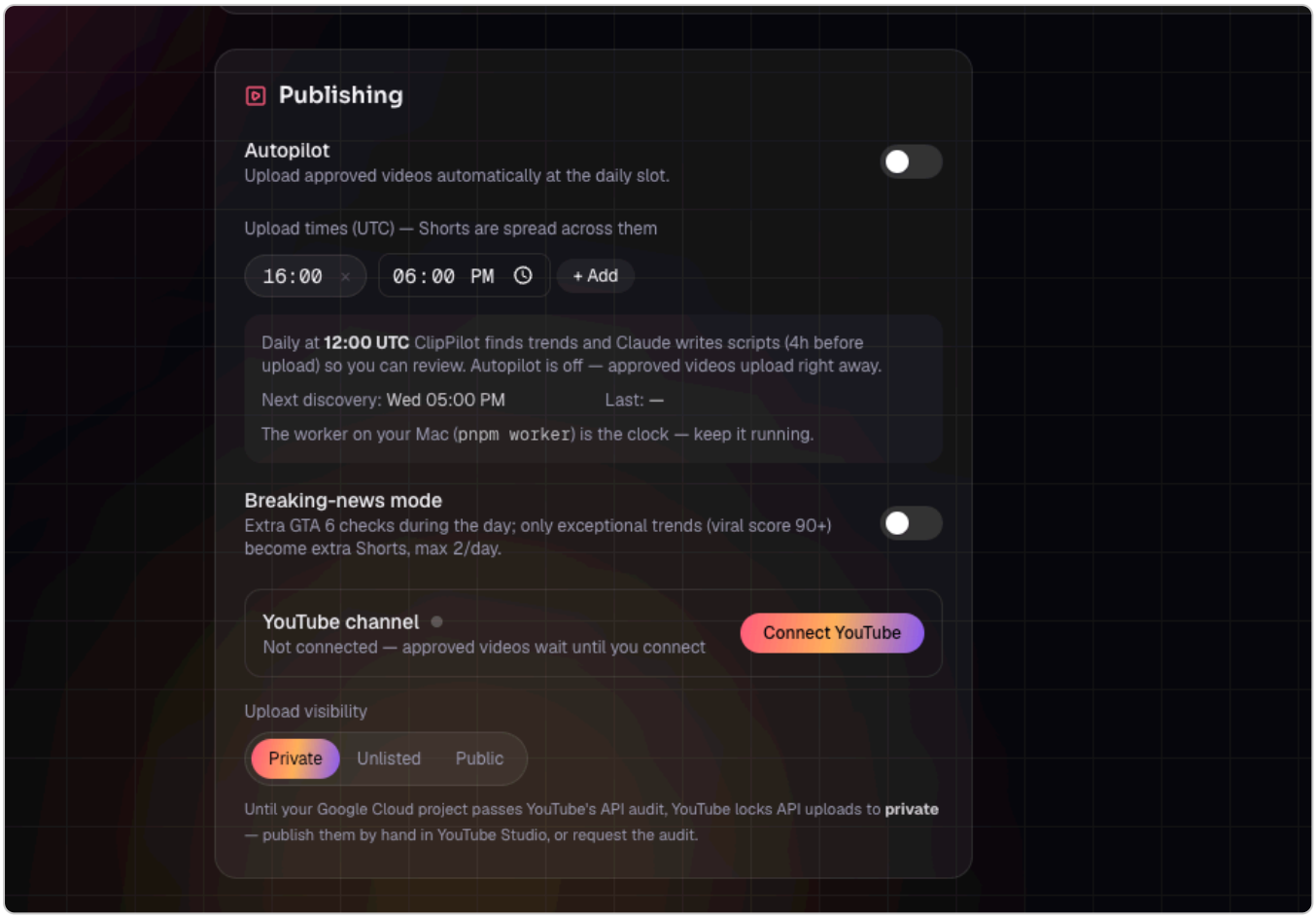


Google consent screen after clicking Connect YouTube: scopes youtube, yt-analytics.readonly, youtube.force-ssl.

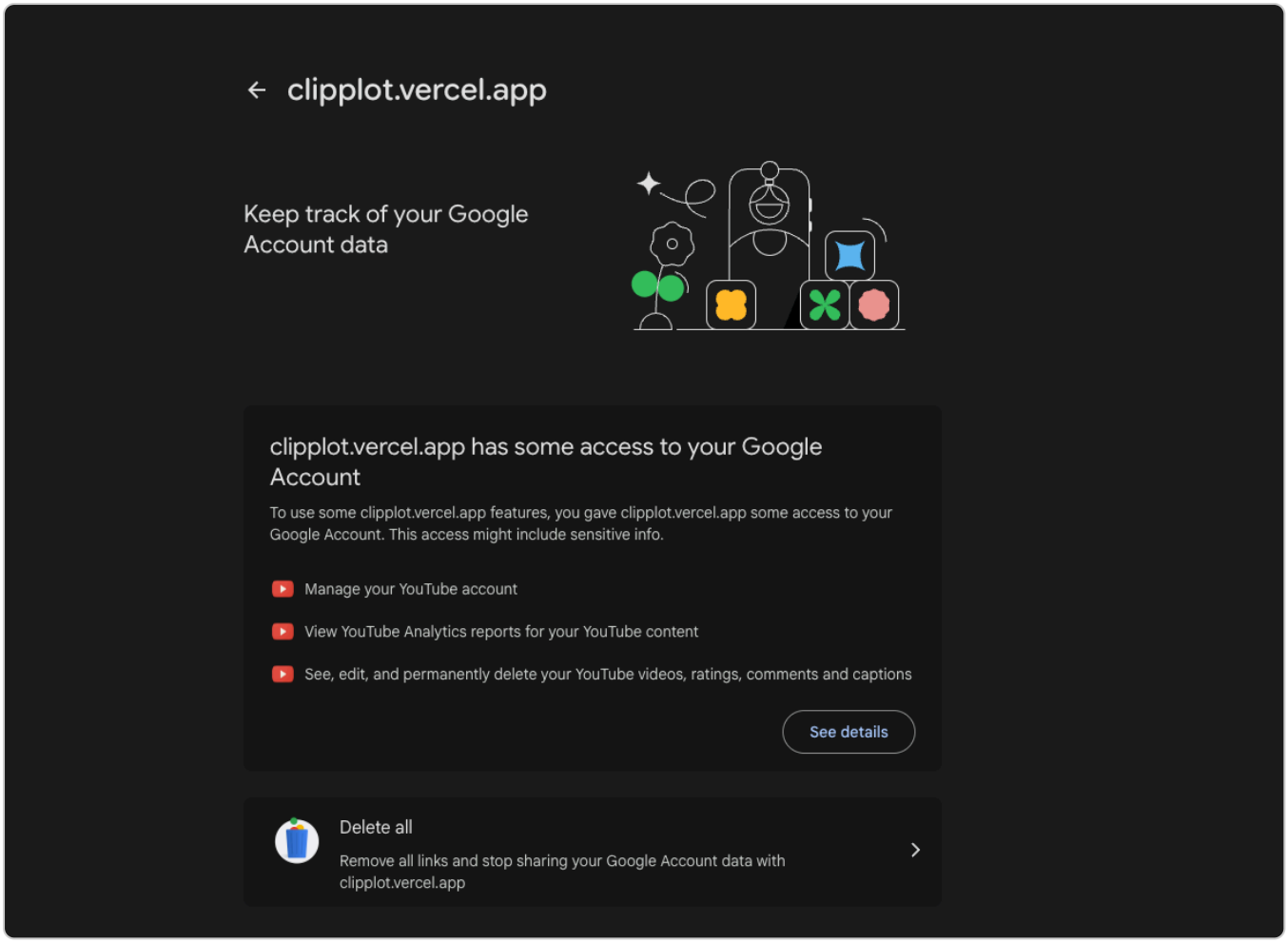


ClipPilot Settings after consent: channel connected, with a Disconnect button.

a) OAuth flow — revocation

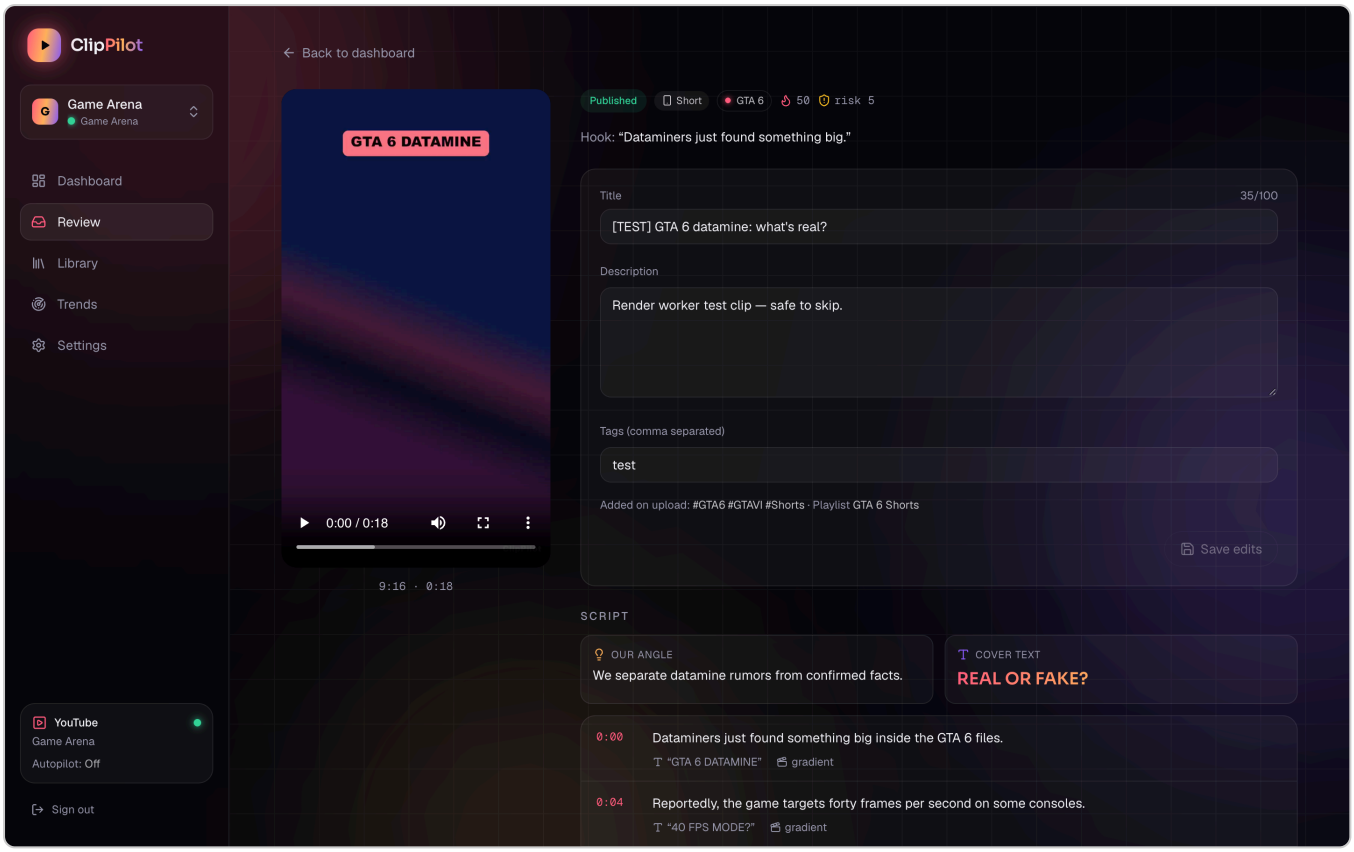


After Disconnect: ClipPilot revokes the token with Google and deletes it; channel shows Not connected.

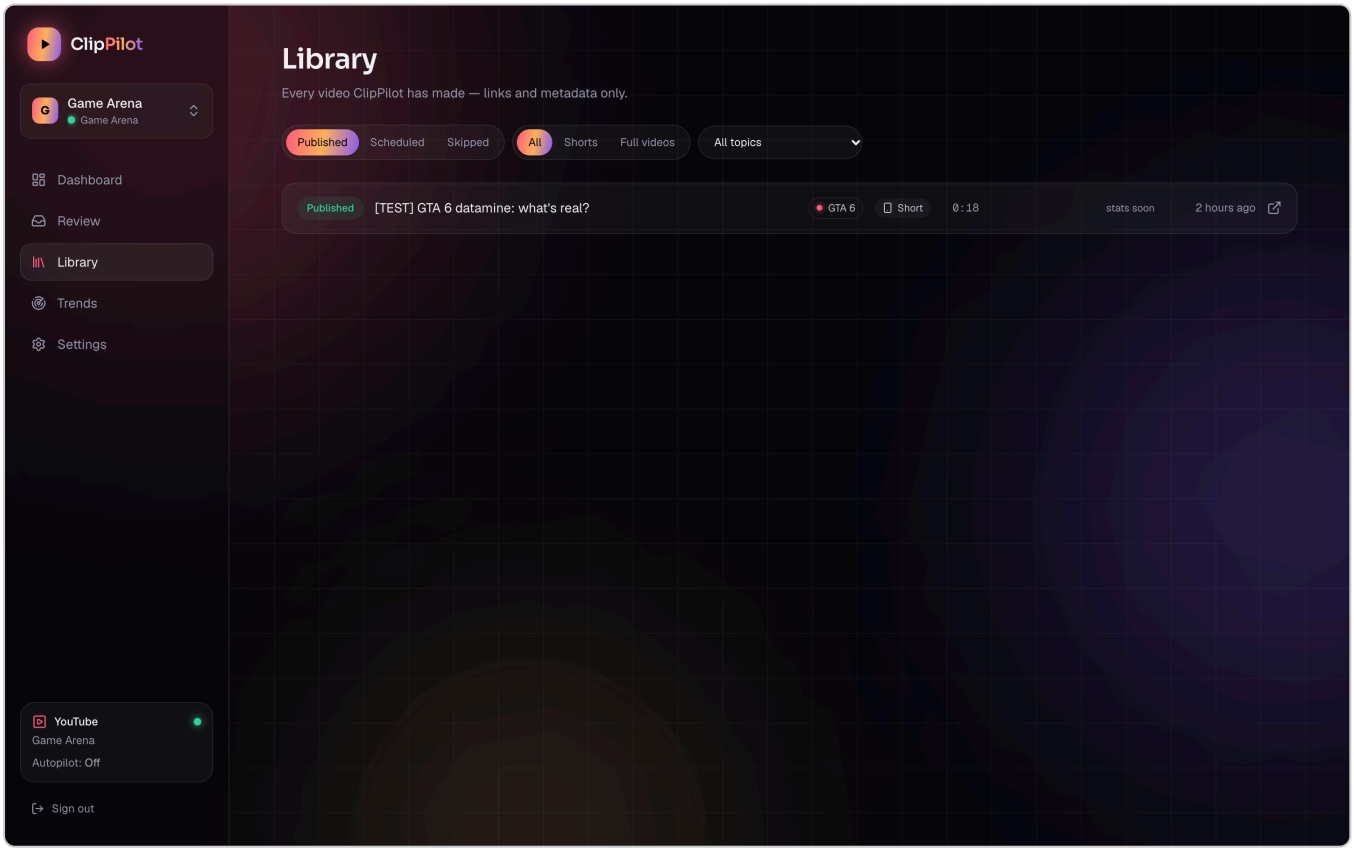


Users can also revoke access in Google Account → Linked apps (myaccount.google.com/connections).

b) Upload interface

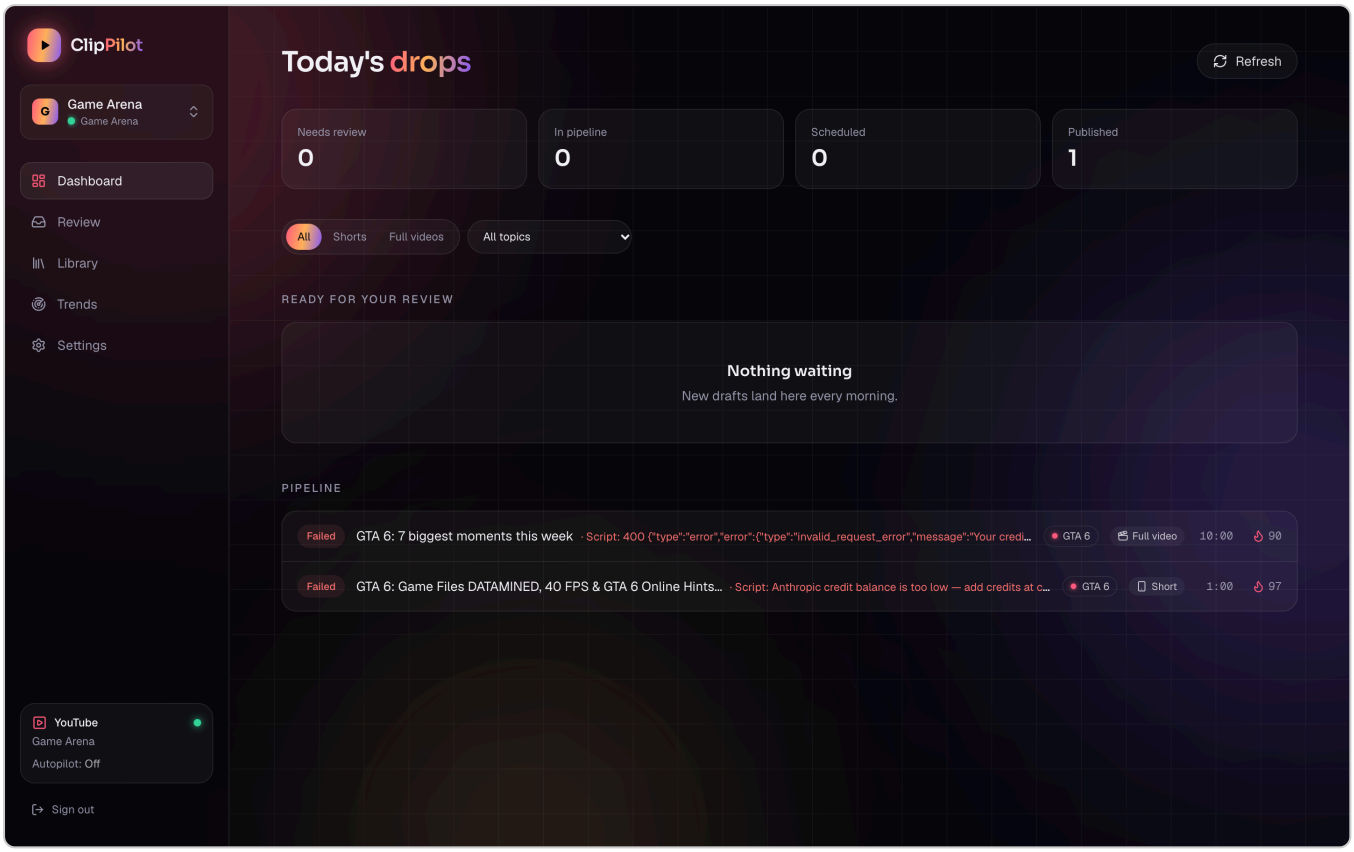


Review screen: preview of our rendered video, editable title / description / tags, playlist and hashtags added on upload.

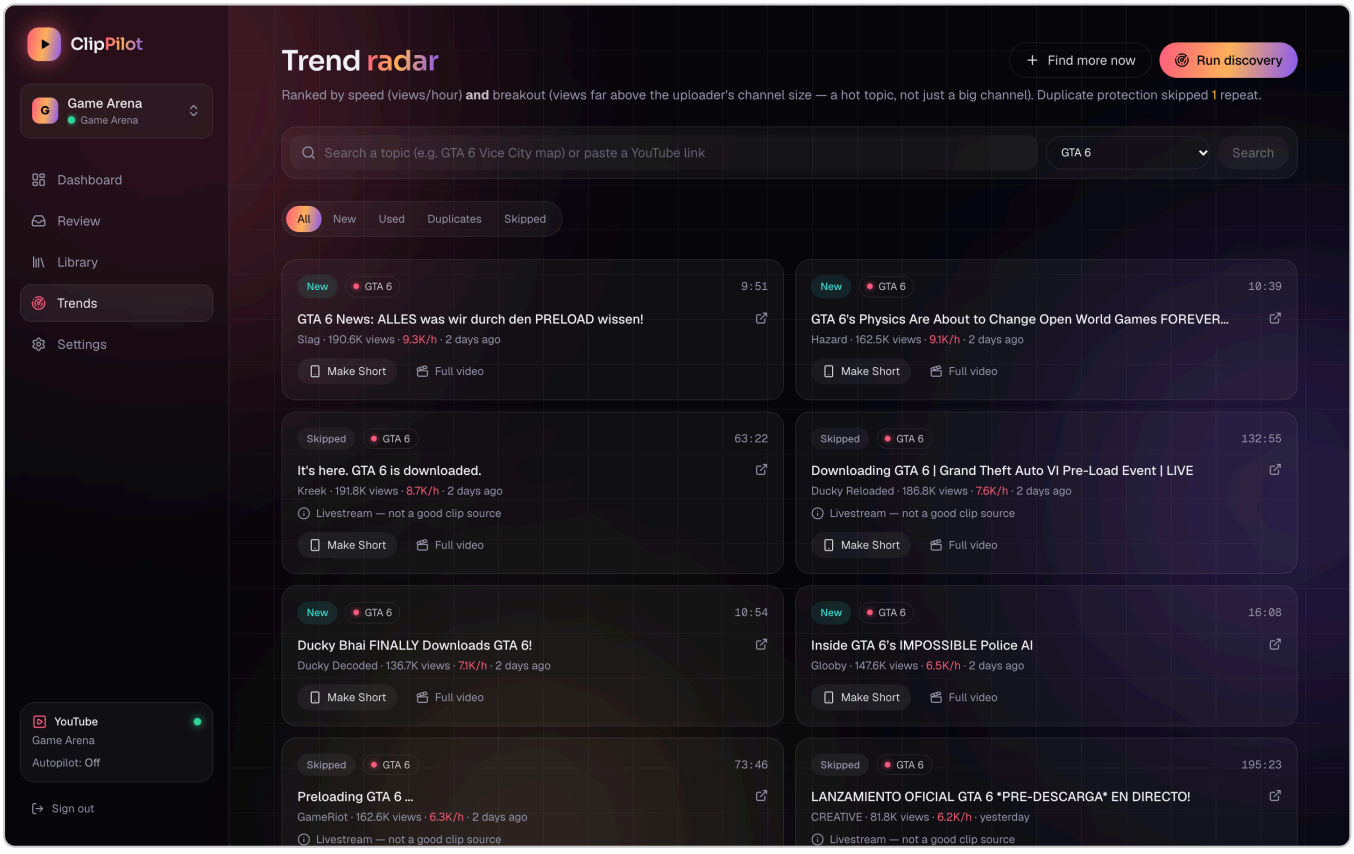


Library: uploaded videos with their YouTube links (links and metadata only).

d) Dashboard and features

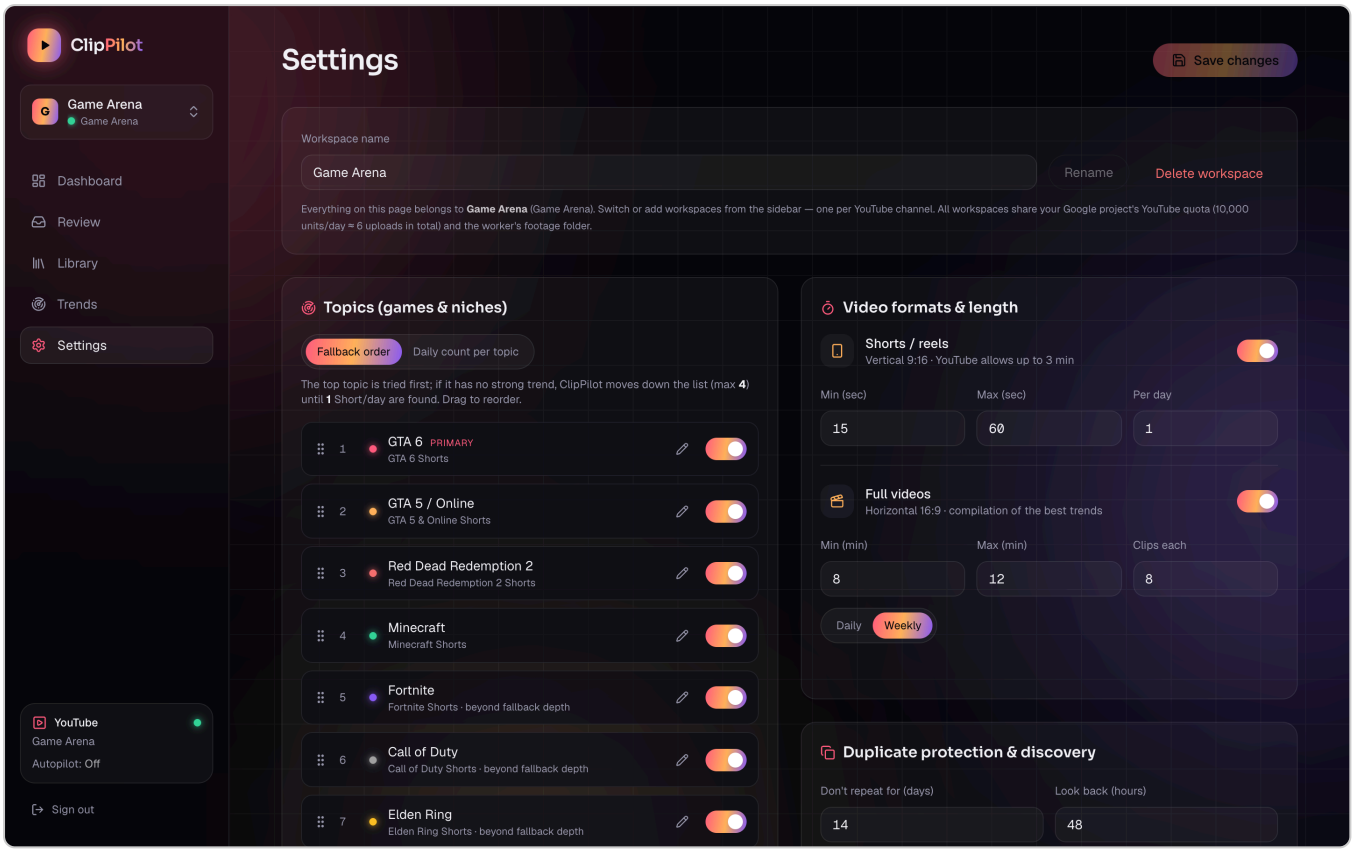


Dashboard: drafts awaiting review and pipeline status.



Trend research (search.list + videos.list): public video titles, view rates and links only.

d) Settings



Settings: topics, upload times, copyright-safety options, YouTube connection and upload visibility.

Complete upload script — worker/youtube.mts

```
import { open, stat } from "node:fs/promises";

/**
 * YouTube resumable upload (videos.insert, uploadType=resumable).
 * Uploads in 8 MB chunks; on network/5xx errors it asks YouTube how much arrived and resumes.
 */
const INIT =
  process.env.WORKER_YT_UPLOAD_URL || // test hook (scripts/upload-check.mts)
  "https://www.googleapis.com/upload/youtube/v3/videos?uploadType=resumable&part=snippet,status";
const CHUNK = 8 * 1024 * 1024; // must be a multiple of 256 KiB
const MAX_RETRIES = 6;

export interface UploadMetadata {
  snippet: Record<string, unknown>;
  status: Record<string, unknown>;
}

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));

async function errorText(res: Response) {
  const body = (await res.json().catch(() => ({}))) as { error?: { message?: string; errors?: { reason?:
string }[] } };
  const reason = body.error?.errors?.[0]?.reason;
  if (reason === "quotaExceeded") return "YouTube daily upload quota used up — it retries tomorrow";
  if (reason === "uploadLimitExceeded") return "This channel hit YouTube's daily upload limit";
  return `YouTube ${res.status}: ${body.error?.message ?? res.statusText}${reason ? ` (${reason})` : ""}`;
}

export async function uploadToYouTube(
  file: string,
  metadata: UploadMetadata,
  accessToken: string,
  log: (m: string) => void,
): Promise<string> {
  const size = (await stat(file)).size;

  // 1. start a session
  const init = await fetch(INIT, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${accessToken}`,
      "Content-Type": "application/json; charset=UTF-8",
      "X-Upload-Content-Length": String(size),
      "X-Upload-Content-Type": "video/mp4",
    },
    body: JSON.stringify(metadata),
  });
  if (!init.ok) throw new Error(await errorText(init));
  const session = init.headers.get("location");
  if (!session) throw new Error("YouTube didn't return an upload session URL");

  // 2. send chunks
  const fh = await open(file, "r");
  try {
    let offset = 0;
    let retries = 0;
    let lastPct = -1;
    while (offset < size) {
      const len = Math.min(CHUNK, size - offset);
      const buf = Buffer.alloc(len);
      await fh.read(buf, 0, len, offset);
      let res: Response;
      try {
        res = await fetch(session, {
          method: "PUT",
          headers: { "Content-Length": String(len), "Content-Range": `bytes ${offset}-${offset + len -
1}/${size}` },
          body: buf,
        });
      } catch (err) {
        res = new Response(null, { status: 599, statusText: err instanceof Error ? err.message : "network
error" });
      }

      if (res.status === 200 || res.status === 201) {
        const video = (await res.json()) as { id?: string };
        if (!video.id) throw new Error("Upload finished but YouTube returned no video id");
        log(`upload 100% → video ${video.id}`);
        return video.id;
      }
      if (res.status === 308) {
        // "Range: bytes=0-N" = what YouTube has so far
        const range = res.headers.get("range");
        offset = range ? Number(range.split("-")[1]) + 1 : 0;
        retries = 0;
        const pct = Math.floor((offset / size) * 100);
        if (pct >= lastPct + 10) {
```

```
          log(`upload ${pct}%`);
          lastPct = pct;
        }
        continue;
      }
      if (res.status >= 500 || res.status === 599 || res.status === 429) {
        if (++retries > MAX_RETRIES) throw new Error(`Upload kept failing (${res.status}
${res.statusText})`);
        const wait = Math.min(60_000, 1000 * 2 ** retries);
        log(`upload hiccup (${res.status}), retrying in ${wait / 1000}s…`);
        await sleep(wait);
        // ask where to resume
        const probe = await fetch(session, { method: "PUT", headers: { "Content-Range": `bytes */${size}` }
}).catch(() => null);
        if (probe?.status === 200 || probe?.status === 201) {
          const v = (await probe.json()) as { id: string };
          return v.id;
        }
        if (probe?.status === 308) {
          const range = probe.headers.get("range");
          offset = range ? Number(range.split("-")[1]) + 1 : 0;
        }
        continue;
      }
      throw new Error(await errorText(res)); // 4xx: not retryable
    }
    throw new Error("Upload ended without a response from YouTube");
  } finally {
    await fh.close();
  }
}

/** Custom thumbnail (needs a phone-verified channel; otherwise YouTube refuses — non-fatal). */
export async function setThumbnail(videoId: string, jpg: Buffer, accessToken: string): Promise<string |
null> {
  const res = await fetch(`https://www.googleapis.com/upload/youtube/v3/thumbnails/set?videoId=${videoId}`,
{
    method: "POST",
    headers: { Authorization: `Bearer ${accessToken}`, "Content-Type": "image/jpeg" },
    body: new Uint8Array(jpg),
  });
  if (res.ok) return null;
  return errorText(res);
}
```